Практическая работа №11

Создание простых хранимых процедур.

Хранимые процедуры представляют, по существу, предварительно откомпилированные программы, которые хранятся на узле сервера (и известны серверу). Клиент обращается к хранимой процедуре с помощью механизма вызова удаленных процедур (Remote Procedure Call — RPC).
Поэтому, в частности, потери в производительности, связанные с обработкой данных на уровне записей в системе "клиент/сервер", могут быть частично компенсированы за счет создания подходящих хранимых процедур, позволяющих выполнить обработку данных непосредственно на узле сервера. В клиентском интерфейсе можно вызывать процедуру с помощью ранее описанных скриптов, вызывать процедуры могут пользователи различных привелегий.

Хранимые процедуры позволяют скрыть от пользователя множество специфических особенностей СУБД и базы данных и благодаря этому достичь более высокой степени независимости от данных по сравнению с тем случаем, когда хранимые процедуры не используются.

- Одна хранимая процедура может совместно использоваться многими клиентами.
- Оптимизация может быть осуществлена при создании хранимой процедуры, а не во время выполнения. (Безусловно, это преимущество проявляется лишь в тех системах, в которых оптимизация обычно осуществляется во время выполнения.)
- Хранимые процедуры позволяют обеспечить более высокую степень безопасности данных. Например, некоторому пользователю может быть разрешено вызывать определенную процедуру, но не разрешено

непосредственно обрабатывать данные, к которым он может иметь доступ через эту хранимую процедуру.

Синтаксис хранимой процедуры:

```
DELIMITER |
    CREATE PROCEDURE NAME_PROCEDURE (IN NAME_VAR TYPE_VAR,
    OUT NAME_VAR TYPE_CAR)
    BEGIN
    …
    END |
    DELIMITER ;
```

Синтаксис для вызова хранимой процедуры:

```
CALL NAME_PROCEDURE (‘VALUE_VAR’, @NAME_VAR1, @NAME_VAR2);
SELECT @NAME_VAR1, @NAME_VAR2;
```

Где:

По умолчанию MySQL использует точку с запятой (;) в качестве разделителя операторов. Это помогает гарантировать, что клиент посылает оператор на сервер целиком, не смешивая его с другими операторами. Однако составной оператор в хранимой процедуре может включать один или более разделителей в добавок к финальному разделителю определения, эти разделители могут вызвать путаницу при передаче оператора CREATE PROCEDURE с клиента на сервер. Чтобы разрешить эту проблему, MySQL поддерживает использование оператора DELIMITER, который позволяет временно изменить разделитель для передачи всего определения процедуры на сервер как единого оператора.

NAME_PROCEDURE – уникальное имя процедуры

В скобках указывается, как значение соответствие после оператора IN, так и значение выходных данных с помощью оператора OUT. Процедуры возможно создавать и без них.

Вызов процедуры возможен с помощью оператора CALL, если в процедуре использовались выходные параметры, то необходимо также вызвать их с помощью оператора SELECT. Обращение к созданным переменным осуществляется с помощью @.

Пример 1. Создание простейших процедур без входных параметров.

В примере создаются и вызываются 2 простейшие процедуры без входных параметров. Первая формирует выборку из связанных таблиц, вторая также формирует выборку из связанных таблиц и использует внутри вложенный запрос.

Обе процедуры вызываются просто по уникальному идентификатору с помощью оператора CALL. Хранятся процедуры во вкладке «процедуры».

```
DELIMITER |
CREATE PROCEDURE get_plane_info()
BEGIN
  SELECT passengers.name_pass, register.price, register.id_avia, city.name_city
  FROM passengers INNER join register on passengers.id_pass=register.id_passenger
  inner join avia on register.id_avia=avia.id_avia
  inner join city on avia.id_city=city.id_city
  where passengers.age>18;
END |
DELIMITER ;
```

Рисунок 1 – Создание простейшей процедуры

```
call get_plane_info();
```

Рисунок 2 – Вызов простейшей процедуры

```
DELIMITER |
create PROCEDURE get_city_info()
BEGIN
select city.name_city, count(avia.id_city), sum(register.price)
FROM city INNER JOIN avia on city.id_city=avia.id_city
INNER JOIN register on avia.id_avia=register.id_avia
group by city.name_city
HAVING sum(register.price)>(select min(register.price)
                            from register);
end |
DELIMITER ;
```

Рисунок 3 – Создание простейшей процедуры с подзапросом

```
call get_city_info()
```

Рисунок 4 – Вызов простейшее процедуры **Пример 2.**

Создание простейших процедур на обновление.

В примере создана процедура на обновление значений в таблице, если остались неопределенные ранее строки. Вызывается такая процедура аналогичным образом. В качестве входных параметров не принимает никаких значений.

```
DELIMITER |
create procedure update_price()
begin
update register inner join passengers on register.id_passenger=passengers.id_pass
inner join avia on avia.id_avia=register.id_avia
inner join city on avia.id_city=city.id_city
SET register.price=CASE
when age<2 then 0
when age>=2 and age<18 then city.distance*2-city.distance*2*0.4
when age>=18 then city.distance*2
end
where register.price is null;
end |
DELIMITER ;
```

Рисунок 5 – Создание простейшей процедуры на обновление

```
call update_price()
```

Рисунок 6 – Вызов простейшей процедуры

Пример 3. Создание процедур с входными и выходными параметрами.

В данной процедуре используется оператор IN во входных параметрах, что говорит о том, что необходимо при вызове передать заданное значение,

которое в дальнейшем отфильтрует записи по этому значению. В текущем примере формируется выборка по городам, количеству рейсов и сумме, которую заплатили пассажиры за билеты. На входе ожидается значение города, которое передается в процедуру при вызове, об этом также говорит строка кода WHERE city.name_city=name, где значению города приравнивается значение из входного параметра.

```
DELIMITER |
CREATE PROCEDURE get_info_city(in name varchar(50))
BEGIN
select city.name_city, count(avia.id_city), sum(register.price)
FROM city INNER JOIN avia on city.id_city=avia.id_city
INNER JOIN register on avia.id_avia=register.id_avia
where city.name_city=name
group by city.name_city
HAVING sum(register.price)>(select min(register.price)
                            from register);
end
```

Рисунок 7 – Создание процедуры с входным параметром.

```
call get_info_city('Москва')
```

Рисунок 8 – Вызов процедуры с входным параметром.

В данном случае во время вызова было передано значение соответствующего города. Входных параметров может быть несколько, но все они должны быть обозначены в дальнейшем в блоке WHERE.

В этом примере описано создание процедуры с входными и выходными параметрами. В качестве входного параметра используется, как и в предыдущем примере, название города, а в качестве выходных имя пассажира и количество рейсов, которое он произвел.

Различие в синтаксисе состоит в том, что после оператора выборки добавляется оператор INTO, который отображает какие переменные будут использованы в качестве выходных данных, блок WHERE остается неизменным, т.к. используется входной параметр.

Для того, чтобы запрос вернул значения мало одного оператора CALL с указанными входными и выходными данными. Помимо этого необходимо вызвать SELECT с соответсвующими выходными параметрами (переменными), вызываемыми и в CALL и в последующем SELECT с помощью знака @

```
DELIMITER |
create PROCEDURE get_pass_info(in name varchar(30),
                               out name_people varchar (30),
                               out count_avia int)
BEGIN
SELECT passengers.name_pass, count(register.id_avia)
into name_people, count_avia
from passengers INNER JOIN register on passengers.id_pass=register.id_passenger
inner join avia on avia.id_avia=register.id_avia
inner join city on avia.id_city=city.id_city
WHERE city.name_city=name;
end |
DELIMITER ;
```

Рисунок 9 – Создание процедуры с входными и выходными параметрами

```
call get_pass_info ('Москва', @name_people, @count_avia);
Select @name_people, @count_avia;
```

Рисунок 10 – Вызов процедуры с входными и выходными параметрами.

Задание.

Создать 2 процедуры без входных и выходных параметров (на выборку и вставку или обновление).

Создать 2 процедуры с входными параметрами и вызвать их.

Создать 2 процедуры с входными и выходными параметрами и вызвать их.